

Übungsblatt 3

Frontend-Technologien

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Lernziele

- **Semantisches HTML** strukturieren und Barrierefreiheit (a11y) berücksichtigen
- **Personas und Nutzerszenarien** für Frontend-Entscheidungen einsetzen
- **CSS-Layouts** mit Flexbox/Grid und **responsive Design** mit Media Queries umsetzen
- **JavaScript** für DOM-Manipulation, fetch und asynchrone Verarbeitung verwenden
- Den Unterschied zwischen **statischem**, **server-rendertem** und **clientseitigem** Frontend einordnen

Modul A — HTML

Übung 3.A.1: Semantisches HTML

Gegeben ist folgendes unsemantisches HTML:

```
<div class="header">
  <div class="nav">
    <div class="nav-item"><a href="/">Home</a></div>
    <div class="nav-item"><a href="/products">Produkte</a></div>
  </div>
</div>
<div class="main">
  <div class="article">
    <div class="title">Produktname</div>
    <div class="text">Beschreibung...</div>
  </div>
</div>
<div class="footer">© 2026</div>
```

1. Schreiben Sie die Struktur mit semantischen HTML5-Elementen um.
2. Warum ist semantisches HTML für Suchmaschinen und Screenreader wichtig?

Übung 3.A.2: Formulare und Accessibility

Erstellen Sie ein barrierefreies HTML-Formular für eine Produktsuche mit:

- Texteingabe für den Suchbegriff (Pflichtfeld)
- Dropdown für Kategorie (Elektronik, Kleidung, Bücher)
- Checkbox „Nur verfügbare Produkte“
- Submit-Button

Welche drei HTML-Attribute sind für Barrierefreiheit unverzichtbar?

Übung 3.A.3: Barrierefreiheit — Problemkategorien und Prinzipien

Barrierefreiheit ist nach EU-Richtlinien ein verpflichtender Aspekt der Web-Entwicklung. Barrieren werden in drei Kategorien klassifiziert: **Wahrnehmungs-**, **Verständnis-** und **Zugriffsprobleme**.

1. Nennen und erklären Sie zu jeder Kategorie zwei Beispiele. Welche Kategorie ist hauptsächlich technischer Natur?
2. Was verbirgt sich hinter dem **Selbstbestimmungsprinzip**, dem **Zwei-Sinne-Prinzip** und dem **Universal-Design-Prinzip**?
3. Nennen Sie ein Prüftool für Barrierefreiheit. Ist die Schriftfarbe `#ffffff` auf dem Hintergrund `#0aae00` barrierefrei?

Übung 3.A.4: Personas — Entertainmentsystem im Flugzeug

Sie unterstützen einen Flugzeughersteller bei der Entwicklung eines In-Flight-Entertainmentsystems mit Touchscreens in den Sitzlehnen der ersten Klasse. Diskutiert werden verschiedene Bedienmethoden (Touch, Tastaturpad, Armlehnenarmatur). Auch unterschiedliche technische Affinität von Passagieren und Personal soll berücksichtigt werden.

1. Was ist eine **Persona** im Kontext der Anforderungserhebung, und welche Felder sollte sie typischerweise enthalten?
2. Entwerfen Sie eine **primäre** und eine **sekundäre** Persona für das System.
3. Beschreiben Sie für jede Persona ein **Szenario**, in dem sie das Entertainmentsystem benutzt.

Modul B — CSS

Übung 3.B.1: Box-Modell und Selektoren

Gegeben folgendes CSS:

```
.card {  
  width: 300px;  
  padding: 20px;  
  border: 2px solid black;  
  margin: 16px;  
}
```

1. Wie breit ist `.card` im Standard-Box-Modell (content-box)? Begründen Sie.
2. Wie breit wäre `.card` mit `box-sizing: border-box`?
3. Schreiben Sie einen CSS-Selektor, der nur das erste `.card`-Element innerhalb eines `<section>` trifft.

Übung 3.B.2: Responsive Layout mit Flexbox

Implementieren Sie ein responsives Produktraster:

- Desktop ($\geq 768\text{px}$): 3 Spalten nebeneinander, gleichmäßig verteilt
- Mobile ($< 768\text{px}$): 1 Spalte

Verwenden Sie ausschließlich Flexbox (kein Grid). Schreiben Sie das vollständige CSS für den Container `.product-grid` und die Karten `.product-card`.

Übung 3.B.3: CSS-Selektoren lesen und schreiben

Gegeben ist folgendes HTML einer TODO-Liste:

```
<h1>Aufgaben:</h1>
<p>Folgende Aufgaben müssen <strong>unbedingt</strong> erledigt werden.</p>
<ul>
  <li class="styles" for="basic" id="entry01">Aufräumen</li>
  <li>Einkaufen</li>
  <li for="basicStudies">Lernen
    <ol>
      <li id="entry02">HTML</li>
      <li>CSS</li>
    </ol>
  </li>
  <li for="peace">Entspannen</li>
</ul>
```

Schreiben Sie CSS-Regeln für folgende Anforderungen (jeweils ein Selektor):

1. Alle Listeneinträge **ohne** die Klasse `styles` bekommen Schriftgröße 18 px.
2. Das **zweite** Element jeder Liste (TODO und Unterliste „Lernen“) wird ausgeblendet.
3. Der Text „unbedingt“ wird rot gefärbt.
4. Der „Aufgaben:“-Header wird um 10° an der Z-Achse gedreht und um 20 px nach rechts und 40 px nach unten verschoben.
5. Alle Listeneinträge mit `for="basic"` werden durchgestrichen.
6. Wenn der Bildschirm schmaler als 600 px ist, wird die Schriftgröße im body auf 10 px reduziert.

Modul C — JavaScript

Übung 3.C.1: DOM-Manipulation und Events

Implementieren Sie folgende Funktion in JavaScript:

Ein Button `#toggle-theme` soll beim Klick die Klasse `dark-mode` auf dem `<body>` togglen. Zusätzlich soll der Button-Text zwischen „Dark Mode“ und „Light Mode“ wechseln.

Schreiben Sie den vollständigen JavaScript-Code (kein Framework).

Übung 3.C.2: Fetch API und asynchrones JavaScript

Implementieren Sie eine Funktion `loadProduct(id)`, die:

1. Die URL `https://api.example.com/products/{id}` aufruft (GET)
2. Das JSON-Ergebnis in ein `<div id="product-detail">` rendert (Felder: `name`, `price`, `description`)
3. Fehler (Netzwerk oder HTTP-Status ≥ 400) mit einer Fehlermeldung im gleichen `<div>` anzeigt

Verwenden Sie `async/await`.

Modul D — Praktische JavaScript- Aufgaben

Übung 3.D.1: Luhn-Algorithmus — Kreditkarten-Validierung

Der **Luhn-Algorithmus** prüft, ob eine Kreditkartennummer formal korrekt ist (Tipp-fehler-Erkennung).
Algorithmus:

1. Letzte Ziffer = Prüfziffer, nicht verändern
2. Von rechts: jede zweite Ziffer verdoppeln
3. Wenn das Ergebnis ≥ 10 : beide Stellen addieren (z.B. $16 \rightarrow 1+6 = 7$)
4. Alle Ziffern aufsummieren
5. Wenn die Summe durch 10 teilbar ist $\rightarrow$ gültig

Implementieren Sie eine JavaScript-Funktion `isValidLuhn(number)`, die:

- Eine Kreditkartennummer als String entgegennimmt
- `true` zurückgibt wenn die Nummer den Luhn-Check besteht, sonst `false`

Testen Sie mit: 4532015112830366 (gültig) und 1234567890123456 (ungültig)



Übung 3.D.2: Web Workers und Local Storage

1. **Web Workers:** Ein Online-Shop berechnet Produktempfehlungen per komplexem Scoring-Algorithmus. Nach der Implementierung friert die UI kurz ein. Erklären Sie: a) Warum friert die UI ein? b) Welche Technologie löst dieses Problem, und wie funktioniert die Kommunikation zwischen UI und dem ausgelagerten Code?
2. **Lokale Datenspeicherung:** Ein Nutzer füllt ein mehrstufiges Registrierungsformular aus. Er schließt versehentlich den Tab. Nennen Sie zwei JavaScript-APIs für clientseitige Persistenz und vergleichen Sie:
 - Lebensdauer (bis wann gehen Daten verloren?)
 - Speicherkapazität (ungefähr)
 - Typischer Einsatz

Übung 3.D.3: Währungsrechner mit Fetch und exchangerate.host

Entwickeln Sie eine HTML-Seite, die Beträge von Euro in andere Währungen umrechnet:

- Eingabefeld für den **Betrag** in Euro
- Auswahlmenü für die **Zielwährung** (USD, CHF)
- Ein „Umrechnen“-Button, der das Ergebnis in einem <output>-Element anzeigt
- Live-Wechselkurse von api.exchangerate.host (kostenlose REST-API)
- Das letzte Ergebnis soll im `localStorage` persistiert und beim erneuten Laden angezeigt werden

Welche **Caching-Strategie** für die Wechselkurse halten Sie für sinnvoll? Begründen Sie.

Materialien zum Übungsblatt

Zum Üben mit den HTML/CSS/JS-Aufgaben stehen vorbereitete Vorlagen bereit:

- [luhn-raw.html](#) — leeres Gerüst für die Luhn-Prüfziffer
- [luhn.html](#) + [luhn.js](#) — funktionierende Referenzlösung
- [currency-api-empty.html](#) — Startgerüst für den Währungsrechner
- [currency-api.html](#) — vollständiger Währungsrechner mit fetch
- [media-query.html](#) — Beispiel für responsives Layout
- [exercise-04.html](#) — kombinierte HTML/JS-Übung
- [datenuebertragung.png](#), [persona.png](#) — Abbildungen aus den Folien

*Alle Materialien finden Sie auch zentral auf der **Kursseite** unter Materials → Download.*